Los entresijos de Git

Arquitectura

- Git es un sistema de archivos direccionable por contenido (*content-addressable*): una base de datos clave-valor donde la clave es el hash SHA-1 del contenido.
- Comandos de bajo nivel (*plumbing*) y de alto nivel (*porcelain*, como add y commit).

Objetos (.git/objects)

- **blob:** contenido de un archivo, sin nombre ni metadatos.
- **tree:** directorio que asocia nombres y permisos con *blobs* y otros *trees*.
- **commit:** apunta a un *tree* raíz y a sus padres, y contiene autor, fecha y mensaje.
- **tag:** referencia anotada a otro objeto.
- Comandos de inspección: git hash-object -w, git cat-file -p y -t, y git ls-tree.

Referencias

- Se almacenan en .git/refs/ (ramas, etiquetas y remotos) como archivos que contienen un hash.
- HEAD es una referencia simbólica a la rama activa.
- Los *refspecs* definen la correspondencia entre referencias remotas y locales en fetch y push.

Almacenamiento y transferencia

- Los objetos sueltos se comprimen con zlib. git gc los consolida en *packfiles* con compresión delta, que guarda las diferencias entre versiones similares.
- La transferencia utiliza *packfiles* para enviar solo los objetos que el otro extremo no posee.
- Los protocolos *dumb* (HTTP estático) y *smart* (pares de procesos como send-pack/receive-pack y fetch-pack/upload-pack).

Mantenimiento y recuperación

- git fsck verifica la integridad y detecta objetos inaccesibles.
- git prune elimina los objetos no referenciados, y git reflog junto con git fsck permiten recuperar datos antes de que se purguen.
- Es posible eliminar objetos grandes del historial, con las consecuencias de reescritura ya descritas.

Variables de entorno

- GIT_DIR, GIT_WORK_TREE, GIT_AUTHOR_NAME y GIT_COMMITTER_NAME, entre otras, modifican el comportamiento y la identidad registrada.

Referencia

S. Chacon y B. Straub, *Pro Git*, 2.ª ed. Apress, 2014. [En línea]. Disponible en: https://git-scm.com/book/es/v2